# PROJECT REPORT

EXPERIMENT - 01

Project title:

IOT-BASED HVAC TEMPERATURE AND HUMIDITY MONITORING SYSTEM

Submitted by:

Arundeepak S -24MER004

Deepak P -24MER006

Dharanidharan R S -24MER007

Dinesh k -24MER009

Vishnuvarthan P -24MER063

## 1. Project Overview

This project implements an IoT-based environmental monitoring system that continuously measures temperature and humidity in a zone and raises a local alert (buzzer + LED) whenever readings move outside a safe, configurable range. It is built as a foundation module of a larger HVAC (Heating, Ventilation, and Air Conditioning) monitoring and energy-management system, where real-time zone data can be used to optimize HVAC operation, reduce energy waste, and flag abnormal conditions before they become costly or unsafe.

The prototype was built and debugged end-to-end on a NodeMCU (ESP8266) development board with a DHT11 temperature/humidity sensor, a low-level-trigger buzzer module, and a status LED, with cloud connectivity added via the ThingSpeak IoT platform for remote data logging and visualization.

## 2. Problem Statement

HVAC systems are among the largest energy consumers in residential and commercial buildings, often accounting for 40–50% of total building energy use. Conventional HVAC control relies on a single thermostat reading or a fixed schedule, which does not reflect real conditions across different zones of a building. This leads to:

- Energy waste from over-cooling/over-heating zones that do not need it
- Delayed detection of abnormal temperature or humidity conditions (e.g., sensor/damper faults, mold-risk humidity levels)
- No remote visibility into environmental conditions for facility managers or homeowners

There is a need for a low-cost, easily deployable IoT sensing unit that can continuously monitor temperature and humidity, provide an immediate local alert when conditions go out of range, and optionally upload data to the cloud for remote monitoring and trend analysis.

## 3. Proposed Solution

A single-zone IoT monitoring node built around a NodeMCU (ESP8266) microcontroller was proposed and implemented with the following capabilities:

- Continuous temperature and humidity sensing using a DHT11 sensor, sampled every 2 seconds
- Local threshold-based alerting: an active buzzer and LED trigger immediately when temperature or humidity exceeds configured safe limits
- Serial diagnostics for real-time debugging and verification during development
- Cloud data logging to ThingSpeak over Wi-Fi, enabling remote visualization of temperature and humidity trends

## 4. System Architecture

The system follows a layered IoT architecture:

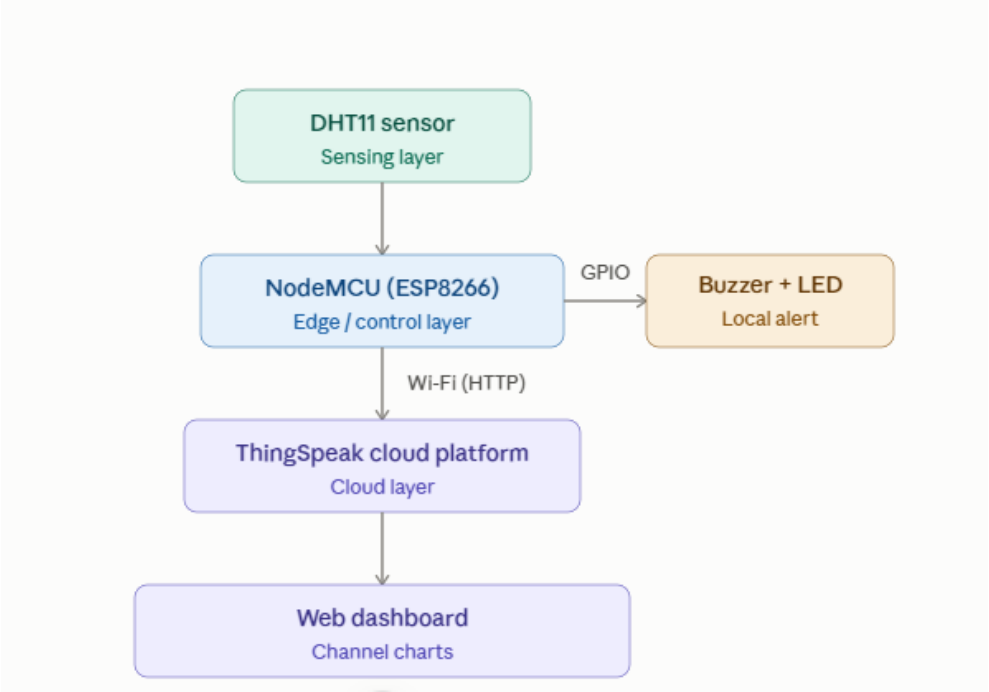


**Fig. 1 Layered IoT system architecture**

- **Sensing Layer**: DHT11 sensor captures ambient temperature and humidity.
- **Edge/Control Layer**: NodeMCU reads the sensor, evaluates threshold logic, and drives the buzzer and LED directly — this works even without network connectivity.
- **Connectivity Layer**: ESP8266 Wi-Fi radio connects to a local network/hotspot (2.4 GHz only).
- **Cloud Layer**: Sensor readings are pushed to a ThingSpeak channel (Channel ID 3455092) via HTTP GET requests to the ThingSpeak Update API.
- **Visualization Layer**: ThingSpeak's built-in channel charts display live and historical temperature/humidity trends.

## 5. Circuit Table

| Component | Pin | NodeMCU Pin | GPIO |
|---|---|---|---|
| DHT11 Sensor | VCC | 3V3 | – |
| DHT11 Sensor | DATA | D4 | GPIO2 |
| DHT11 Sensor | GND | G | – |
| Buzzer Module (low-level trigger) | VCC | 3V3 | – |
| Buzzer Module (low-level trigger) | I/O (Signal) | D5 | GPIO14 |
| Buzzer Module (low-level trigger) | GND | G | – |
| Status LED | Anode (+) via 220–330Ω resistor | D6 | GPIO12 |
| Status LED | Cathode (−) | G | – |

Note: RST, TX/RX, GPIO0 and GPIO15 were deliberately kept free of any connections, as these are reserved for boot-mode selection and reset control; connecting a sensor line to RST during initial prototyping was identified and corrected as a wiring fault during development (see Section 11).

## 6. Circuit Design

The circuit is a single-board prototype centered on the NodeMCU ESP8266. Power is supplied via USB (5V), which is regulated on-board to 3.3V for the sensor, buzzer, and LED. Design considerations:

- DHT11 uses a single digital data line (GPIO2) with the sensor module's built-in pull-up resistor.
- The buzzer module is a 3-pin, low-level-trigger type: driving its signal pin LOW turns it ON, HIGH turns it OFF (opposite of a typical active-high peripheral).
- The LED is driven through a current-limiting resistor (220–330Ω) to prevent excess current draw from the GPIO pin.
- A known ESP8266 power-transient issue was identified during development: the Wi-Fi radio draws a brief current spike immediately after boot/reset, which can brown out a marginal power supply. The recommended long-term hardware fix is a 470–1000µF electrolytic capacitor placed across the 3V3 and GND pins to smooth this transient.

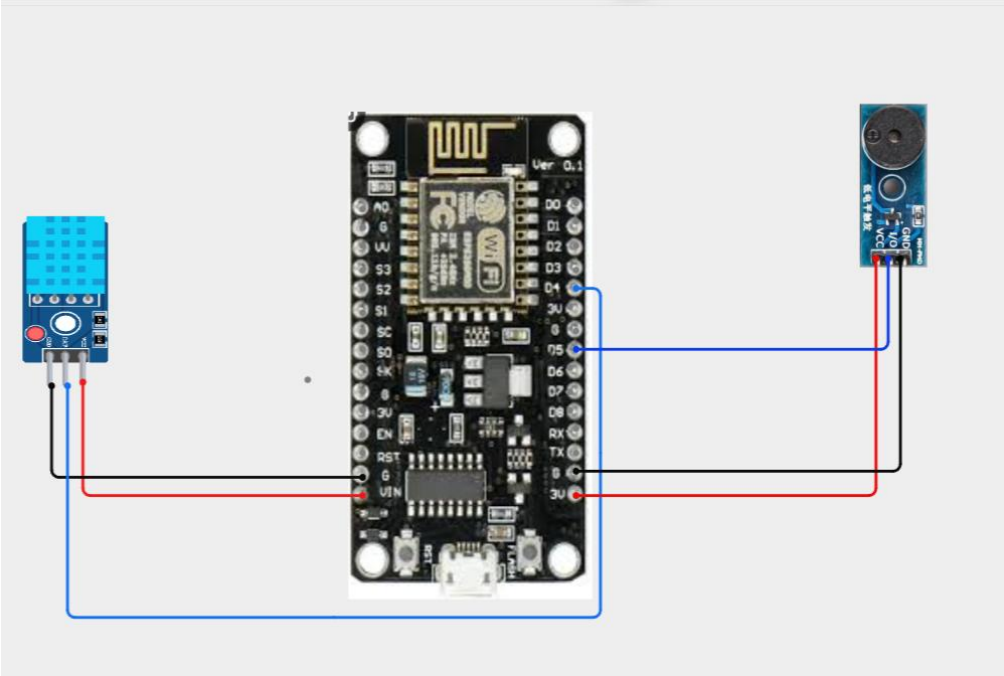


**Fig. 2 Circuit wiring diagram**

## 7. Algorithm

Main control loop logic (executes continuously once the board boots):

1. Initialize serial output, buzzer pin, LED pin, and DHT sensor.
2. Sound a brief startup beep and LED flash to confirm wiring/polarity.
3. Every 2 seconds, read temperature and humidity from the DHT11.
4. If the read fails (NaN), log an error, silence the buzzer/LED, and skip this cycle.
5. Evaluate: alarmActive = (temp ≥ TEMP_MAX) OR (temp ≤ TEMP_MIN) OR (humidity ≥ HUMIDITY_MAX).
6. Set the LED ON if alarmActive is true, OFF otherwise.
7. If alarmActive, pulse the buzzer ON/OFF every 300 ms (audible alert pattern); otherwise keep it silent.
8. (Cloud-enabled variant) Every 20 seconds, if Wi-Fi is connected, upload the last valid reading to the ThingSpeak channel via an HTTP GET request; otherwise skip and retry on the next interval.
9. Repeat indefinitely.

## 8. Coding

The final working firmware (DHT11 + buzzer + LED alert, standalone version):

cpp

```
#include <DHT.h>

/* ---------------------- CONFIGURATION ------------------------- */
#define DHTPIN        2         // GPIO2  (D4)
#define DHTTYPE       DHT11
#define BUZZER_PIN    14        // GPIO14 (D5)
#define LED_PIN       12        // GPIO12 (D6)

#define BUZZER_ON     LOW
#define BUZZER_OFF    HIGH

#define TEMP_MAX_C        30.0
#define TEMP_MIN_C        16.0
#define HUMIDITY_MAX_PCT  75.0

#define READ_INTERVAL_MS  2000UL
#define BUZZER_BEEP_MS     300UL

DHT dht(DHTPIN, DHTTYPE);

unsigned long lastReadMs   = 0;
unsigned long lastBeepMs   = 0;
bool          buzzerOn     = false;
bool          alarmActive  = false;

void setup() {
```

```
  Serial.begin(115200);
  delay(200);

  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, BUZZER_OFF);
  digitalWrite(LED_PIN, LOW);

  digitalWrite(BUZZER_PIN, BUZZER_ON);
  digitalWrite(LED_PIN, HIGH);
  delay(200);
  digitalWrite(BUZZER_PIN, BUZZER_OFF);
  digitalWrite(LED_PIN, LOW);

  dht.begin();
  Serial.println("DHT alert circuit ready.");
}

void loop() {
  unsigned long now = millis();

  if (now - lastReadMs >= READ_INTERVAL_MS) {
    lastReadMs = now;
    float humidityPct = dht.readHumidity();
    float tempC       = dht.readTemperature();

    if (isnan(humidityPct) || isnan(tempC)) {
      Serial.println("[ERROR] Failed to read from DHT sensor");
      alarmActive = false;
      digitalWrite(BUZZER_PIN, BUZZER_OFF);
```

```
      digitalWrite(LED_PIN, LOW);
      return;
    }

    Serial.printf("Temp: %.1f C   Humidity: %.1f %%\n", tempC, humidityPct);

    alarmActive = (tempC >= TEMP_MAX_C) ||
                  (tempC <= TEMP_MIN_C) ||
                  (humidityPct >= HUMIDITY_MAX_PCT);

    digitalWrite(LED_PIN, alarmActive ? HIGH : LOW);

    if (alarmActive) {
      Serial.println("  -> ALERT: reading out of safe range!");
    }
  }

  if (alarmActive) {
    if (now - lastBeepMs >= BUZZER_BEEP_MS) {
      lastBeepMs = now;
      buzzerOn = !buzzerOn;
      digitalWrite(BUZZER_PIN, buzzerOn ? BUZZER_ON : BUZZER_OFF);
    }
  } else if (buzzerOn) {
    buzzerOn = false;
    digitalWrite(BUZZER_PIN, BUZZER_OFF);
  }
}
```

A cloud-connected variant of this firmware was also developed, which re-enables the Wi-Fi radio and adds an HTTP GET call to the ThingSpeak Update API

(http://api.thingspeak.com/update) every 20 seconds, posting temperature to Field 1 and humidity to Field 2 of ThingSpeak Channel ID 3455092.

## 9. System Working

On power-up, the NodeMCU initializes all peripherals and sounds a short confirmation beep with an LED flash. It then enters a continuous sensing loop:

- Every 2 seconds, the DHT11 is polled for temperature and humidity.
- Valid readings are printed to the Serial Monitor for live diagnostics (e.g., "Temp: 28.5 C Humidity: 71.3 %").
- If either reading exceeds the configured safe range (Temp ≥ 30°C, Temp ≤ 16°C, or Humidity ≥ 75%), the system enters an alert state: the LED turns solidly on and the buzzer pulses audibly every 300 ms.
- The system automatically clears the alert and silences the buzzer/LED once readings return to the safe range.
- In the cloud-enabled configuration, the board also connects to a Wi-Fi access point (2.4 GHz only — confirmed necessary during testing) and pushes the latest reading to ThingSpeak every 20 seconds, where it appears as live charts on the channel dashboard.

## 10. Bill of Materials

| Component | Quantity | Purpose |
|---|---|---|
| NodeMCU ESP8266 (WROOM/ESP-12E) | 1 | Main microcontroller with built-in Wi-Fi |
| DHT11 Temperature/Humidity Sensor | 1 | Environmental sensing |
| Buzzer module (3-pin, low-level trigger) | 1 | Audible alert |
| LED (5mm, any color) | 1 | Visual alert indicator |
| Breadboard | 1 | Prototyping platform |
| Jumper wires (M-M) | ~10 | Connections |
| Micro-USB cable (data-capable) | 1 | Programming and power |
| 470–1000µF electrolytic capacitor (recommended) | 1 | Smooths Wi-Fi boot current spike / prevents brownout |
| USB power source (PC port or 5V/1A+ wall adapter) | 1 | Power supply |

## 11. Prototype Implementation

The prototype was assembled and debugged iteratively, with the following notable steps and issues resolved during implementation:

- Initial circuit review identified a wiring fault: a DHT sensor lead was connected to the RST pin, which would force the board into a continuous reset loop. This was corrected by moving the DHT data line to GPIO2 (D4).
- The buzzer's ground lead was initially routed to the TX pin (used for serial communication/boot); this was corrected to a dedicated GND pin.
- Development environment setup required installing the "DHT sensor library" and "Adafruit Unified Sensor" library via the Arduino Library Manager.
- Board profile was set to "Generic ESP8266 Module," which required using raw GPIO numbers (GPIO2, GPIO14) in code instead of Arduino D-pin aliases (D4, D5), since those aliases are only defined under NodeMCU-specific board profiles.
- A recurring "MD5 of file does not match data in flash" upload error was resolved by lowering the upload speed from 115200 to 57600 baud, after ruling out flash size misconfiguration and USB cable/port issues.
- A USB driver mismatch (device alternating between CP210x- and CH340-style identification) was resolved by installing the correct CH340 driver, after diagnosing via Windows Device Manager.
- A post-boot brownout was identified: the board would flash correctly (verified hash) but go silent immediately after reset, caused by the ESP8266's Wi-Fi radio calibration current spike exceeding what the USB power source could supply. This was mitigated in software (forcing Wi-Fi off at boot for the standalone version) and the standard hardware fix (a bulk capacitor across 3V3/GND) was documented for full reliability.
- A Wi-Fi connection failure (status code 1, network not found) was traced to an SSID mismatch caused by a space in the actual hotspot name ("Mr. j") not present in the configured SSID string ("Mr.j"); corrected by matching the SSID exactly, confirmed via a network-scan diagnostic sketch.
- A subsequent linker error (undefined references to vPortFree, pvPortMalloc, etc.) appeared once Wi-Fi functionality was introduced, identified as a known compatibility issue with the installed ESP8266 board package version, with a board-package downgrade and lwIP variant change documented as fixes.
- A buzzer silence issue during confirmed alert conditions was isolated using a minimal direct-drive test sketch and a direct-to-ground hardware test, to separate a possible polarity/logic issue from a wiring or component fault.

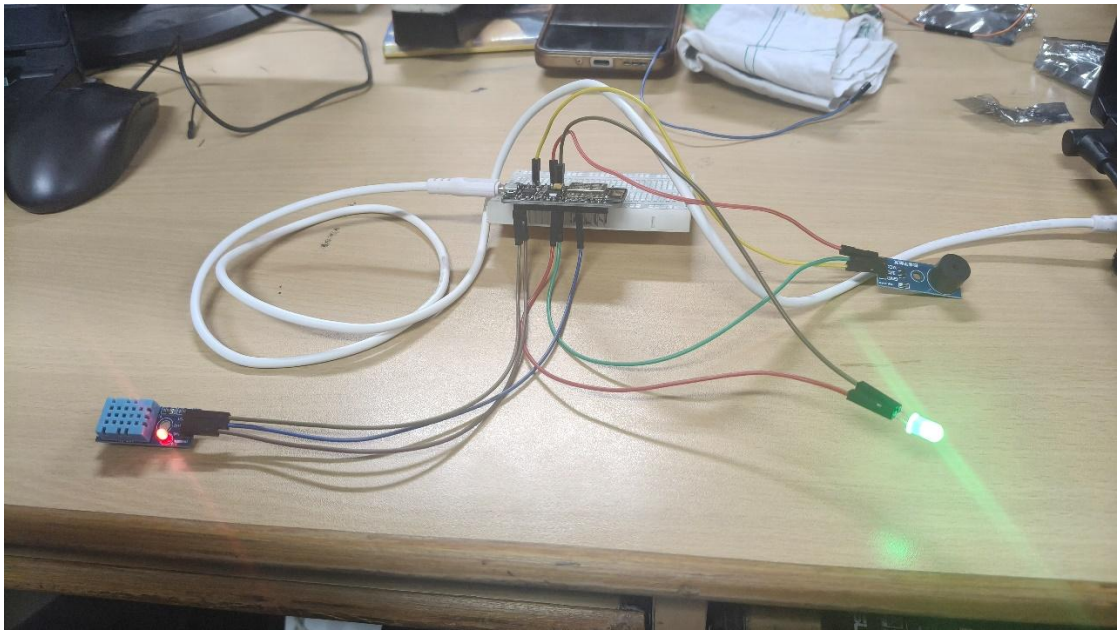

**Fig. 3 Assembled prototype — DHT11, buzzer, LED wired to NodeMCU**

## 12. Results & Performance Analysis

Once wiring and configuration issues were resolved, the system performed as designed:

- The DHT11 sensor produced consistent temperature and humidity readings every 2 seconds, visible on the Serial Monitor (e.g., 28.5°C / 71.3% RH during indoor testing).
- The alert logic correctly triggered when humidity exceeded the configured 70–75% threshold, printing "ALERT: reading out of safe range!" and activating the LED reliably.
- An elevated temperature reading (~40°C) was observed during testing in an air-conditioned room; this was diagnosed as likely due to sensor proximity to the NodeMCU's onboard voltage regulator (a heat source) rather than a true ambient reading, illustrating the importance of physical sensor placement away from heat-generating components.
- Wi-Fi connectivity was successfully established after correcting the SSID string, confirmed via a clean "CONNECTED!" status with an assigned IP address.
- The buzzer's audible alert was not consistently confirmed as working at the time of this report; a direct hardware test (grounding the signal line directly) was prescribed as the next diagnostic step to isolate a wiring/power issue from a defective buzzer module.
- Cloud upload to ThingSpeak (Channel 3455092) was implemented and configured to respect the platform's 15-second minimum update interval, using a 20-second upload cycle; end-to-end cloud verification (data appearing on the channel dashboard) is a pending confirmation step.

Overall, the prototype demonstrates a working local sensing-and-alert pipeline with a clear path to full cloud-connected operation. Remaining work includes final buzzer hardware verification, confirming stable long-duration Wi-Fi/cloud operation (ideally with the capacitor-based brownout fix applied), and extending the system to multiple zones with HVAC actuation for closed-loop energy management.

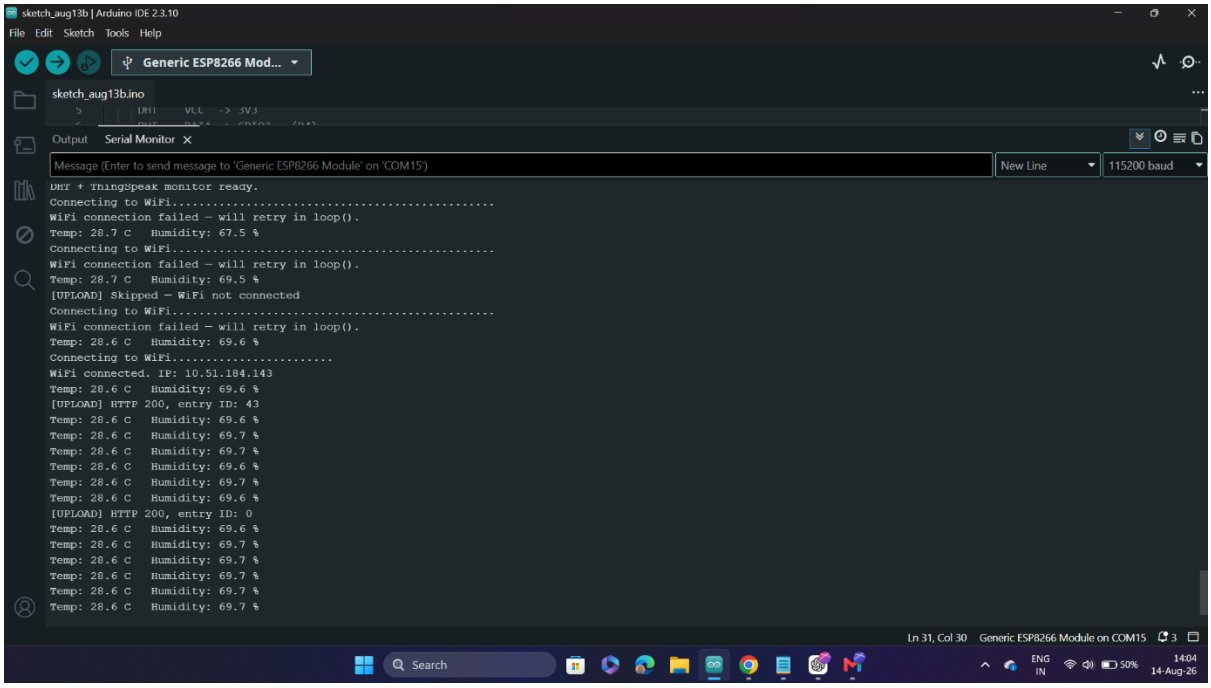


**Fig. 5 Serial Monitor — Wi-Fi connection sequence**

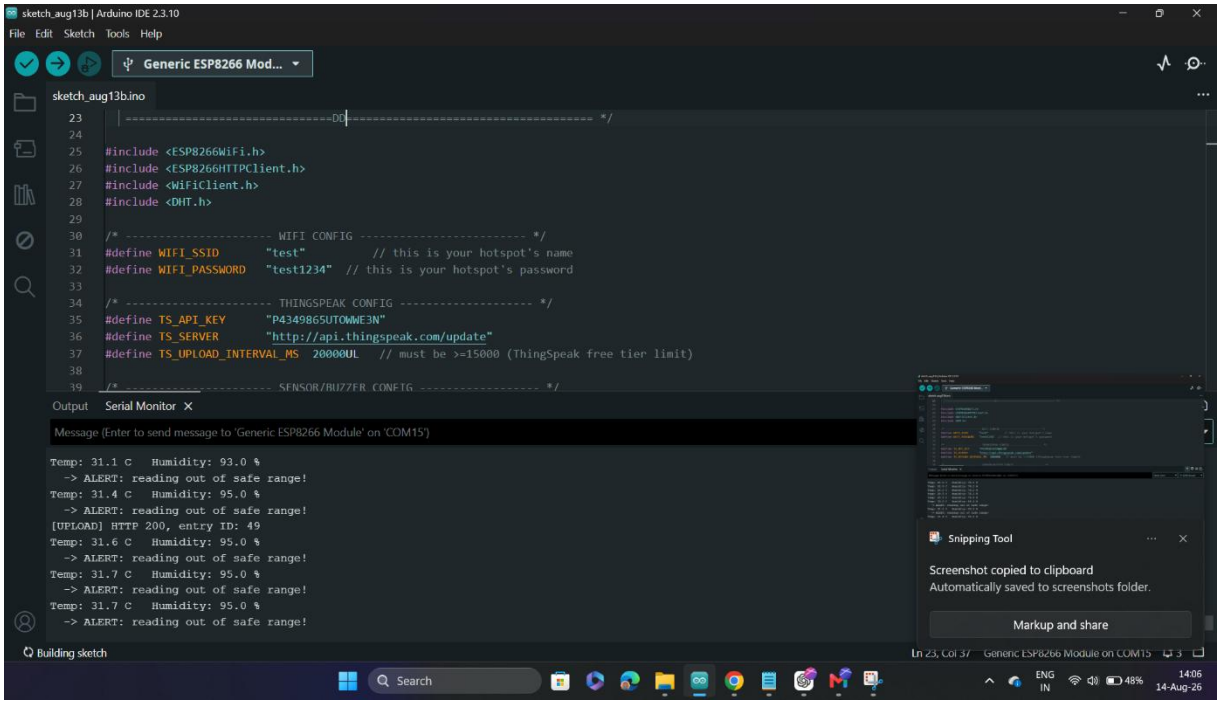


**Fig. 6 Serial Monitor — alert triggers and ThingSpeak upload log**